

SQL Joins File Explanation (Joins.sql)

SQL Code

```
-- =====
-- 1. USER WORKOUT HISTORY
-- Tables: user_profile + training_records
-- Purpose: Show all workouts for a specific user
-- =====
SELECT
    u.user_id,
    u.name,
    u.email,
    tr.record_id,
    tr.date,
    tr.sets,
    tr.reps,
    tr.weight_used,
    tr.duration_minutes,
    tr.notes
FROM user_profile u
INNER JOIN training_records tr ON u.user_id = tr.user_id
ORDER BY tr.date DESC;

-- =====
-- 2. EXERCISE DETAILS WITH EQUIPMENT
-- Tables: exercises + available equipments
-- Purpose: Show exercise information with required equipment
-- =====
SELECT
    e.exercise_id,
    e.exercise_name,
    e.muscle_group,
    e.difficulty_level,
    eq.equipment_id,
    eq.equipment_name,
    eq.equipment_type,
    e.instructions
FROM exercises e
INNER JOIN available equipments eq ON e.equipment_id = eq.equipment_id
ORDER BY e.muscle_group, e.exercise_name;

-- =====
-- 3. COMPLETE WORKOUT DETAILS
-- Tables: training_records + user_profile + exercises
```

```

-- Purpose: Show detailed workout information with user and exercise names
-- =====
SELECT
    tr.record_id,
    u.user_id,
    u.name as user_name,
    u.email,
    e.exercise_id,
    e.exercise_name,
    e.muscle_group,
    e.difficulty_level,
    tr.date,
    tr.sets,
    tr.reps,
    tr.weight_used,
    tr.duration_minutes,
    tr.notes
FROM training_records tr
INNER JOIN user_profile u ON tr.user_id = u.user_id
INNER JOIN exercises e ON tr.exercise_id = e.exercise_id
ORDER BY tr.date DESC, u.name;

-- =====
-- 7. EXERCISE POPULARITY ANALYSIS
-- Tables: exercises + training_records
-- Purpose: Show how popular each exercise is among users
-- =====
SELECT
    e.exercise_id,
    e.exercise_name,
    e.muscle_group,
    e.difficulty_level,
    COUNT(tr.record_id) as times_performed,
    COUNT(DISTINCT tr.user_id) as unique_users,
    AVG(tr.weight_used) as avg_weight,
    SUM(tr.sets * tr.reps) as total_reps
FROM exercises e
LEFT JOIN training_records tr ON e.exercise_id = tr.exercise_id
GROUP BY e.exercise_id, e.exercise_name, e.muscle_group, e.difficulty_level
ORDER BY times_performed DESC;

-- =====
-- 8. COMPLETE WORKOUT WITH EQUIPMENT
-- Tables: training_records + user_profile + exercises + available equipments
-- Purpose: Show complete workout details including equipment information
-- =====
SELECT
    tr.record_id,

```

```

    u.user_id,
    u.name as user_name,
    u.email,
    e.exercise_id,
    e.exercise_name,
    e.muscle_group,
    e.difficulty_level,
    eq.equipment_id,
    eq.equipment_name,
    eq.equipment_type,
    tr.date,
    tr.sets,
    tr.reps,
    tr.weight_used,
    tr.duration_minutes,
    tr.notes
FROM training_records tr
INNER JOIN user_profile u ON tr.user_id = u.user_id
INNER JOIN exercises e ON tr.exercise_id = e.exercise_id
INNER JOIN available equipments eq ON e.equipment_id = eq.equipment_id
ORDER BY tr.date DESC, u.name;

```

This file demonstrates different types of **JOINS**, how they are used, and what each query is meant to show.

1. USER WORKOUT HISTORY

Tables: `user_profile` (alias `u`) + `training_records` (alias `tr`)

INNER JOIN

This join returns only the users who have at least one matching workout record.

`u.user_id = tr.user_id` identifies which workouts belong to which user.

ORDER BY

Sorts the results by the workout date in descending order (latest workout first).

2. EXERCISE DETAILS WITH EQUIPMENT

Tables: `exercises` (alias `e`) + `available equipments` (alias `eq`)

INNER JOIN

Shows only exercises that have equipment linked to them.

Displays exercise name, muscle group, difficulty level, and its required equipment.

ORDER BY

Sorts by muscle group and exercise name alphabetically.

3. COMPLETE WORKOUT DETAILS (User + Exercise + Workout)

Tables: `training_records` (`tr`), `user_profile` (`u`), `exercises` (`e`)

Multiple INNER JOINS

Collects combined details of: - user performing the workout - exercise performed - workout stats (sets, reps, weight, duration)

ORDER BY

Sorted by workout date (latest first), and then by user name.

7. EXERCISE POPULARITY ANALYSIS

Tables: `exercises` (`e`) + `training_records` (`tr`)

LEFT JOIN

Shows all exercises, even those that have **never been performed**.

Aggregation Functions:

- `COUNT(tr.record_id)` → times an exercise was performed
- `COUNT(DISTINCT tr.user_id)` → how many unique users performed it
- `AVG(tr.weight_used)` → average weight used

- `SUM(tr.sets * tr.reps)` → total reps performed for that exercise

GROUP BY

Groups data by exercise so aggregates apply per exercise.

ORDER BY

Sorts exercises by popularity (most performed first).

8. COMPLETE WORKOUT WITH EQUIPMENT

Tables: `training_records (tr)`, `user_profile (u)`, `exercises (e)`,
`available equipments (eq)`

Chained INNER JOINS

Connects all four tables: - Workout details - User information - Exercise details - Equipment used for that exercise

Produces a full combined record for each workout.

ORDER BY

Sorted by workout date (newest first), then by user name.